Battery dispatch

Deterministic and stochastic formulations

Guilherme Machado

18 October 2026

Abstract

The two formulations of the battery dispatch problem in one document: the deterministic mixed-integer program solved over 2018–2020 with perfect foresight, and its multistage stochastic counterpart over a scenario tree. Part I states the model, its solution approach and the rolling-window decomposition; Part II drops the assumption that prices are known in advance and states the policy problem that replaces it.

Contents

I Battery dispatch model **3**

1 Problem statement **5**

2 Notation **6**
2.1 Sets and indices . . . 6
2.2 Parameters . . . 6
2.3 Decision variables . . . 6

3 Objective **8**

4 Constraints **9**
4.1 Storage bounds . . . 9
4.2 Energy balance . . . 9
4.3 Complementarity Constraint: charge or discharge, not both . . . 9
4.4 Capacity constraints . . . 10
4.5 Market 2 hourly commitment . . . 10
4.6 Cycle life . . . 10
4.7 Storage volume degradation . . . 11

5 Assumptions and simplifications **12**

6 Solution approach **13**
6.1 Solver . . . 13
6.2 Problem tractability . . . 13
6.3 The decomposition algorithm . . . 13
6.4 What decomposition into windows costs . . . 14
6.5 Lagrangian relaxation of the cycle budget . . . 14
6.6 Minimising the dual: the cutting-plane method . . . 15
6.7 The cycle budget on this data . . . 16
6.8 How much foresight is enough . . . 17

7 Results **18**

II Battery dispatch under stochastic prices **19**

1 What changes, and what does not **21**

2 Uncertainty and information **22**
2.1 Probability space and filtration . . . 22
2.2 The scenario tree . . . 22

2.3 Who decides what, and when 23

3 Notation **24**

4 Objective **25**
4.1 Expected profit 25
4.2 The objective collapses onto conditional mean prices 25

5 Constraints **27**
5.1 Storage bounds 27
5.2 Energy balance within a day 27
5.3 State handover at a day boundary 27
5.4 Complementarity and rate limits 27
5.5 Market 2 hourly commitment 28
5.6 Cycle accounting and cycle life 28
5.7 Storage volume degradation 28
5.8 Terminal condition 28

6 The deterministic equivalent, and its size **29**
6.1 What a decomposition would have to respect 30

7 Special cases and checks **31**

8 Assumptions this formulation adds **32**

9 The scenario form **33**

10 What is deferred **34**

Part I

Battery dispatch model

Formulation of the battery dispatch problem for the Aurora second-round technical exercise: battery trading across two wholesale electricity markets over 2018–2020, as a price taker.

Chapter 1

Problem statement

We maximize revenue, a battery buys energy (charges) when prices are low and sells (discharge) when they are high. Two wholesale markets are available, quoted in £/MWh and settled at different time resolutions, and the battery may split its power between them. The objective is to choose a charge/discharge schedule that maximises profit over the horizon.

The battery is a **price taker**: its own actions do not move either market's prices, so the price series are exogenous inputs. They are also taken as **known**: the whole horizon is given before any decision is made. `model-stochastic.md` states the same model with that second assumption dropped.

The problem is posed as a mixed-integer linear program on a **common half-hourly grid**. Market 1 settles natively at that resolution; Market 2 settles hourly, and is represented by holding its decision variables constant across the two half hours of each hour.

Chapter 2

Notation

2.1 Sets and indices

Symbol	Meaning
$t \in \mathcal{T}$	half-hourly settlement period, $t = 1, \ldots, T$
$h \in \mathcal{H}$	trading hour
$\mathcal{T}_h$	the two half-hourly periods making up hour h
$m \in \{1, 2\}$	market

2.2 Parameters

Symbol	Value	Unit	Meaning
Δt	0.5	h	length of a settlement period
$p_1(t)$	—	£/MWh	Market 1 price in period t (half-hourly)
$p_2(t)$	—	£/MWh	Market 2 price in period t (hourly, held across $\mathcal{T}_h$)
$\overline{C}$	2	MW	maximum charge rate, across both markets
$\overline{D}$	2	MW	maximum discharge rate, across both markets
$\overline{S}_{\max}$	4	MWh	nominal storage volume, before degradation
e_c	0.05	–	fraction of imported energy lost before storage
e_d	0.05	–	fraction of discharged energy lost before reaching the grid
$\overline{L}$	5000	cycles	cycle life
ρ	10^{-5}	/cycle	storage volume lost per equivalent full cycle

Parameter values are from Attachment 1. The spreadsheet quotes the degradation rate in percent (0.001 %/cycle), so $\rho = 10^{-5}$ as a fraction.

2.3 Decision variables

Symbol	Domain	Unit	Meaning
$c_1(t)$	≥ 0	MW	charging power bought from Market 1
$c_2(t)$	≥ 0	MW	charging power bought from Market 2
$d_1(t)$	≥ 0	MW	discharging power drawn from storage for Market 1
$d_2(t)$	≥ 0	MW	discharging power drawn from storage for Market 2
$S(t)$	≥ 0	MWh	energy stored
$u(t)$	binary	–	1 when charging, 0 when discharging
$S_{\max}(t)$	≥ 0	MWh	effective storage volume in period t, after degradation
$n_{\text{cyc}}(t)$	≥ 0	cycles	equivalent full cycles accumulated up to period t

The two sides are measured at different points. c_m is power **imported from the grid**, of which $(1 - e_c)\, c_m$ reaches storage. d_m is power **drawn from storage**, of which $(1 - e_d)\, d_m$ reaches the grid, so the store is debited the full d_m in §4.2 while only $(1 - e_d)\, d_m$ is paid for in §3. Each rate limit in §4.3 caps its own variable directly, both being treated as capabilities of the battery.

$n_{\text{cyc}}(t)$ counts whole cycles physically, but is left continuous rather than integer: rounding it up would be conservative and costs an integer variable in an already large model.

Chapter 3

Objective

Maximise total profit over the horizon. Revenue in a period is the energy delivered to each market at that market's price, less the energy bought; the factor $\Delta t = \frac{1}{2}$ converts power in MW to energy in MWh over the half-hourly period. Only $(1 - e_d)$ of the power drawn from storage reaches the grid, and only that part is paid for.

$$\max \sum_{t \in \mathcal{T}} R(t)$$

where

$$\begin{aligned} R(t) &= \frac{p_1(t)(1-e_d)d_1(t) + p_2(t)(1-e_d)d_2(t) - p_1(t) \cdot c_1(t) - p_2(t) \cdot c_2(t)}{2} \\ &= \frac{p_1(t)\left((1-e_d)\, d_1(t) - c_1(t)\right) + p_2(t)\left((1-e_d)\, d_2(t) - c_2(t)\right)}{2} \end{aligned}$$

Chapter 4

Constraints

4.1 Storage bounds

Stored energy stays between empty and the effective storage volume, which is itself a variable, and shrinks over the horizon as the battery degrades (§4.7).

$$0 \le S(t) \le S_{\max}(t) \qquad \forall t \in \mathcal{T}$$

4.2 Energy balance

Energy stored carries from one period to the next, increased by what is charged and reduced by what is discharged in that period. The factor Δt converts power in MW to energy in MWh over the half-hourly period, matching the objective.

Charging is lossy: a fraction e_c of what is imported from the grid is lost before it reaches storage, so only $(1 - e_c)\,(c_1 + c_2)$ is stored. Discharging debits the store by the full $d_1 + d_2$, since d_m is defined at the storage side; the discharge loss is taken in §3, where only $(1 - e_d)\,d_m$ is paid for.

$$S(t) = S(t-1) + \Delta t\,[(1 - e_c)(c_1(t) + c_2(t)) - d_1(t) - d_2(t)] \qquad \forall t \in \mathcal{T}$$

with the battery starting empty:

$$S(0) = 0$$

The round-trip efficiency is $(1 - e_c)(1 - e_d) = 0.9025$, so a price spread of about 11% is needed before a cycle breaks even.

4.3 Complementarity Constraint: charge or discharge, not both

A binary $u(t)$ makes charging and discharging mutually exclusive, and simultaneously imposes the rate limits across both markets.

$$\begin{aligned} 0 \le c_1(t) + c_2(t) &\le \overline{C} \cdot u(t) \\ 0 \le d_1(t) + d_2(t) &\le \overline{D} \cdot (1 - u(t)) \\ c_1(t),\ c_2(t),\ d_1(t),\ d_2(t) &\ge 0 \end{aligned}$$

When $u(t) = 1$ the battery may charge and all discharge is forced to zero; when $u(t) = 0$ the reverse.

Both rates are read as capabilities of the battery itself, so each caps its own variable directly and no efficiency factor appears here. The discharge limit therefore bounds the power drawn from storage, and the most the battery can deliver to the grid in a period is $(1 - e_d)\overline{D} = 1.9$ MW.

4.4 Capacity constraints

Charging is limited by the headroom remaining in the store, and discharging by what the store holds.

$$\Delta t\,(1 - e_c)(c_1(t) + c_2(t)) \le S_{\max}(t) - S(t-1)$$
$$\Delta t\,(d_1(t) + d_2(t)) \le S(t-1)$$

Both are implied by the storage bounds of §4.1 together with the balance of §4.2, so they are redundant; they are stated because they make explicit what the bounds enforce only indirectly.

4.5 Market 2 hourly commitment

Capacity committed to Market 2 is held for that market's entire hourly interval. This applies to selling as much as to buying, so both of its decision variables are constant across the two half hours of each hour.

$$\begin{aligned} c_2(t_1) &= c_2(t_2) \\ d_2(t_1) &= d_2(t_2) \end{aligned} \qquad \forall\, t_1, t_2 \in \mathcal{T}_h, \quad \forall h \in \mathcal{H}$$

Market 1 settles half-hourly and so carries no such coupling.

Note the interaction with §4.3: because $u(t)$ forces all discharge to zero in a period where the battery charges, holding d_2 constant across the hour means the battery cannot charge in one half hour and sell into Market 2 in the other half of the same hour. That is the intended physical behaviour: a commitment to deliver into Market 2 must be honoured for the full hour.

4.6 Cycle life

The brief defines one cycle as charging to the maximum storage volume and then discharging all of it, and allows the energy to be accumulated in parts. The equivalent full cycles used over the horizon are therefore the total energy throughput divided by the storage volume, and that total is capped by the battery's cycle life.

A cycle is only completed when the energy is discharged, so the count is taken on the discharge side alone: charging without discharging consumes no cycle life.

Cycles are accumulated period by period, so the count is available at every t for the degradation in §4.7:

$$n_{\mathrm{cyc}}(t) = n_{\mathrm{cyc}}(t-1) + \frac{\Delta t\,(d_1(t) + d_2(t))}{\overline{S}_{\max}} \qquad \forall t \in \mathcal{T}$$

$$n_{\mathrm{cyc}}(0) = 0$$

$$n_{\mathrm{cyc}}(T) \le \overline{L}$$

The cap is written at the end of the horizon, but $n_{\mathrm{cyc}}(t)$ is non-decreasing, since $d_1, d_2 \ge 0$, so $n_{\mathrm{cyc}}(t) \le n_{\mathrm{cyc}}(T)$ for every t, and the two forms

$$n_{\mathrm{cyc}}(T) \le \overline{L} \qquad \Longleftrightarrow \qquad n_{\mathrm{cyc}}(t) \le \overline{L} \quad \forall t \in \mathcal{T}$$

define the same feasible set. The implementation uses the **first**: one row on the final period, with $n_{\mathrm{cyc}}(t)$ left otherwise unbounded above. The cheaper-looking alternative, the cap as the upper bound of every variable at no cost in rows, makes the cap impossible to *relax*: §6 prices the budget by removing it from the subproblem, and a bound removes nothing. Declaring the variable free keeps one row and one place where the budget is imposed, which is what the pricing methods need. The variable is bounded regardless, through the accumulation above and the rate limits of §4.3.

The equivalence of the two forms rests on monotonicity and on $\overline{L}$ being a single number for the whole horizon. It would fail for a time-varying allowance $\overline{L}(t)$, such as a per-year quota, or for any count that could decrease, and the $\forall t$ form would then be the only correct one.

Throughput is measured at the storage side, matching the brief's definition of a cycle in terms of stored volume: $d_1 + d_2$ is what leaves the store. The nominal $\overline{S}_{\max}$ is the denominator rather than the degraded $S_{\max}(t)$, which keeps the constraint linear.

4.7 Storage volume degradation

Each equivalent full cycle costs a fixed fraction ρ of the storage volume, applied to the cycles accumulated so far, so the usable volume shrinks as the horizon progresses.

$$S_{\max}(t) = \overline{S}_{\max}\,(1 - \rho\, n_{\mathrm{cyc}}(t)) \qquad \forall t \in \mathcal{T}$$

This is linear in $n_{\mathrm{cyc}}(t)$, so introducing it keeps the problem a MILP. At the cycle limit the cumulative loss is $\rho\overline{L} = 5\%$ of the nominal volume.

$S_{\max}(t)$ is a convenience rather than a necessity: it appears only in §4.1 and §4.4, where substituting the expression above turns each bound into a constraint on $S(t)$ and $n_{\mathrm{cyc}}(t)$ directly. An implementation may drop the variable; the write-up keeps it because it names the quantity.

Chapter 5

Assumptions and simplifications

The brief invites simplification over completeness. Choices made here:

- **Price taker.** Stated in the brief; prices are exogenous.
- **Prices are known over the whole horizon.** The schedule is chosen with perfect foresight, which is what makes the result of §7 an upper bound on what a policy without foresight could earn rather than a forecast of revenue. The stochastic formulation is in `model-stochastic.md`.
- **The battery starts empty.** $S(0) = 0$. The balance needs some initial state; the alternatives are to leave $S(0)$ free, or to impose a cyclic condition $S(T) = S(0)$ so the horizon neither starts nor ends with a windfall.
- **The rate limits are battery capabilities, not grid limits.** Attachment 1 describes them as the power the battery can import from and export to the grid, which would put them on the grid side of the converter. They are read instead as limits on the battery's own charge and discharge power, so each bounds its variable directly.
- **The cycle budget is the full 5,000 over a three-year horizon.** The price data covers three of the battery's ten calendar years, so spending the entire cycle life within it implies retiring the asset early. Pro-rating to $\frac{3}{10} \times 5000 = 1500$ cycles would be the alternative. Which is right depends on whether the exercise is "maximise profit over this data" or "value the asset over its life"; the former is what the brief asks for.
- **Capex and opex do not enter the objective.** £500,000 capex and £5,000/year fixed opex are fixed with respect to the dispatch decision, so they do not change the optimal schedule; they belong to a separate economic assessment of the asset.

Chapter 6

Solution approach

6.1 Solver

HiGHS, through `pulp`. HiGHS reads the model in process rather than through a file, which matters because the horizon is solved as a sequence of many small programs rather than one large one.

6.2 Problem tractability

Posed whole, the program carries one binary $u(t)$ for each of the 52,608 periods. Solve time grows steeply, and erratically, with the window length: on this data a day solves in about 0.04 s and two days in 0.2 s, but a week takes around 3 s and a fortnight anywhere from 5 s to 17 s depending on which fortnight it is. The cliff sits between 30 and 60 days: a month solves to proven optimality in 30 s, two months could not be proved in 900 s. Posed whole, the program is 315,649 rows, 368,256 columns and 52,608 binaries; given an hour, HiGHS never left the root node, returning a feasible schedule worth £204,077.77 against a dual bound of £208,209.57, a 2.02% gap and £3,287 worse than the decomposition reaches in minutes. The full measurements are in `tractability.md`.

The horizon is therefore walked in windows. Each window solves commit + lookahead periods, keeps the decisions of the committed part, and hands S and n_{cyc} at that point to the next window as its opening state. One day committed with one day of lookahead is the default. The lookahead is solved and then discarded; without it each day would be decided as if the horizon ended at midnight, and the battery would always end the day empty even when the next morning pays more than the evening did.

6.3 The decomposition algorithm

Write W for the committed length and H for the lookahead, both in periods and both a whole number of hours, so that no hourly interval of §4.5 is split across a window boundary. Let $\mathcal{W}_k = \{kW, \dots, \min(kW + W + H, T)\}$ be the k-th window, and $P(\mathcal{W}, s)$ the program of §3–§4.7 restricted to the periods $\mathcal{W}$ and opened from state $s = (S^0, n^0_{\text{cyc}})$. That is, §4.2 and §4.6 read S^0 and n^0_{cyc} as their $t-1$ values in the window's first period, and §4.6 caps the *cumulative* count, not the window's own use.

$s_0 \leftarrow (0, 0)$
for $k = 0, 1, \ldots, \lceil T/W \rceil - 1$:
$x^k \leftarrow \arg\max P(\mathcal{W}_k, s_k)$ (one MILP)
commit $x^k(t)$ for $t \in \{kW, \ldots, kW + W - 1\}$, discard the lookahead
$s_{k+1} \leftarrow (S^{x^k}(kW + W - 1),\ n_{\text{cyc}}^{x^k}(kW + W - 1))$
return the concatenated committed decisions

- **The only thing lost is foresight.** Each window optimises as if the horizon ended at $kW + W + H$. What that costs is measured below.

6.4 What decomposition into windows costs

Two couplings reach beyond a single window:

- **State of charge.** Carried exactly by the handover, so the only loss is of foresight past the lookahead, not of feasibility.
- **Cycle life (§4.6).** Global. Each window carries the cycles already spent and caps the cumulative count, so the budget can never be overspent. But a window cannot see what a later one would have done with a cycle, so a binding budget is consumed by the earliest days that can use it.

The second is handled by pricing the budget rather than splitting it: a Lagrangian relaxation of the one constraint that couples distant windows.

6.5 Lagrangian relaxation of the cycle budget

Relax $n_{\text{cyc}}(T) \le \overline{L}$ into the objective with a multiplier $\lambda \ge 0$. Writing $R(t)$ for the revenue of §3 and $\mathcal{X}$ for the schedules satisfying all of §4 *except* the cap,

$$L(\lambda) \;=\; \max_{x \in \mathcal{X}} \Big[\sum_{t \in \mathcal{T}} R(t) \;-\; \lambda \big(n_{\text{cyc}}(T) - \overline{L} \big) \Big]$$

and because $\lambda \overline{L}$ is a constant and $n_{\text{cyc}}(T)$ is a sum over windows of what each window spends, the bracket is **separable**: for a fixed λ each window can be solved alone, needing to know nothing about the rest of the horizon except the price. A global constraint has been traded for a per-window charge, which is what the decomposition needs. Implemented, the charge is the $-\lambda \, (n_{\text{cyc}}(T) - n_{\text{cyc}}^0)$ term already written into the objective of each window.

The algorithm rests on three standard properties:

1. L **is convex and piecewise linear in** λ, being a pointwise maximum of functions affine in λ.
2. $\overline{L} - n_{\mathbf{cyc}}(T)$ **is an exact subgradient** of L at λ, evaluated at the maximiser.
3. **Weak duality:** $L(\lambda) \ge P^\star$ **for every** $\lambda \ge 0$, where $P^\star$ is the optimum of the constrained problem. So every evaluation is an upper bound on the best achievable profit, and $\min_{\lambda \ge 0} L(\lambda)$ is the tightest such bound this relaxation offers.

λ is in £ per equivalent full cycle and is an opportunity cost: the profit the horizon expects to forgo elsewhere if this window spends a cycle here. A window then applies a rule that needs no knowledge of the future, *take the trade only if the spread clears* λ, and at the optimal $\lambda^\star$

that price is the marginal value of the last cycle in the budget, i.e. the budget's shadow price. $\lambda^\star = 0$ means the cap is not binding.

6.6 Minimising the dual: the cutting-plane method

L is minimised over $\lambda \geq 0$ by Kelley's cutting-plane method, which needs only what one evaluation already provides. At each pass, the relaxed problem is solved at the current λ_i, giving the value $L(\lambda_i)$ and the subgradient $g_i = \overline{L} - n_{\text{cyc}}(T)$; by convexity

$$L(\lambda) \;\geq\; L(\lambda_i) + g_i\,(\lambda - \lambda_i) \qquad \forall \lambda \geq 0$$

so each pass contributes a **cut**: a linear function lying everywhere below L. The next λ is the minimiser of the cuts' upper envelope, a two-variable linear program with one row per pass,

$$\begin{aligned} \min_{\lambda,\ \theta} \quad & \theta \\ \text{s.t.} \quad & \theta \geq L(\lambda_i) + g_i(\lambda - \lambda_i) \qquad i = 1, \dots, k \\ & 0 \leq \lambda \leq \lambda_{\max} \end{aligned}$$

which is negligible beside a pass over the horizon. The method then brackets the answer from both sides:

- the master's value $\theta_k^\star$ is a **lower** bound on $\min_\lambda L(\lambda)$, the cuts being underestimators;
- the best $L(\lambda_i)$ evaluated so far is an **upper** bound on it, and by weak duality on $P^\star$ as well.

Both bounds, and the two cuts that produce them, on one picture, with λ along the horizontal axis and the value of the dual L up the vertical:

Two passes have been spent, at λ_1 and λ_2. Each returned a value and a subgradient, and the subgradient is the slope of the piece of L the evaluation landed on, so each cut runs along the curve over that piece and continues underneath it past the next kink. The cuts cross below the curve, at λ_3: that is the master's minimiser, the next multiplier to evaluate, and its height $\theta_k^\star$ is the lower bound. The better of the two values already evaluated is the upper bound, and the vertical distance between the two is the gap. Neither bound is the answer yet: λ_3 misses $\lambda^\star$ because two cuts cannot see the shallow pieces of L around its minimum. Evaluating there adds a third cut, which lifts $\theta_k^\star$ and, if the new value is better, lowers the upper bound; the two close on $\min_\lambda L(\lambda)$ from opposite sides.

The figure is a schematic — the axes carry no numbers, and the pieces are invented to make the shape legible. It is drawn by `scripts/plot_cutting_plane.py`, `make figures`.

The loop therefore terminates on a measured gap, $\min_i L(\lambda_i) - \theta_k^\star \leq \text{tol}$, rather than on an iteration count. The gap at exit is a statement about the quality of the answer, not of the price alone.

Two further elements are needed to make it an algorithm that returns a schedule:

- **Primal recovery.** A relaxed pass may overspend the budget, and an infeasible schedule is not an answer. One final pass *with* the cap, priced at the $\lambda^\star$ the dual settled on, produces the deliverable.
- **Stabilisation, for a vector λ.** Kelley's iterates can oscillate, the classical remedy being a proximal or trust-region term, as in a bundle method. A scalar λ did not call for it on

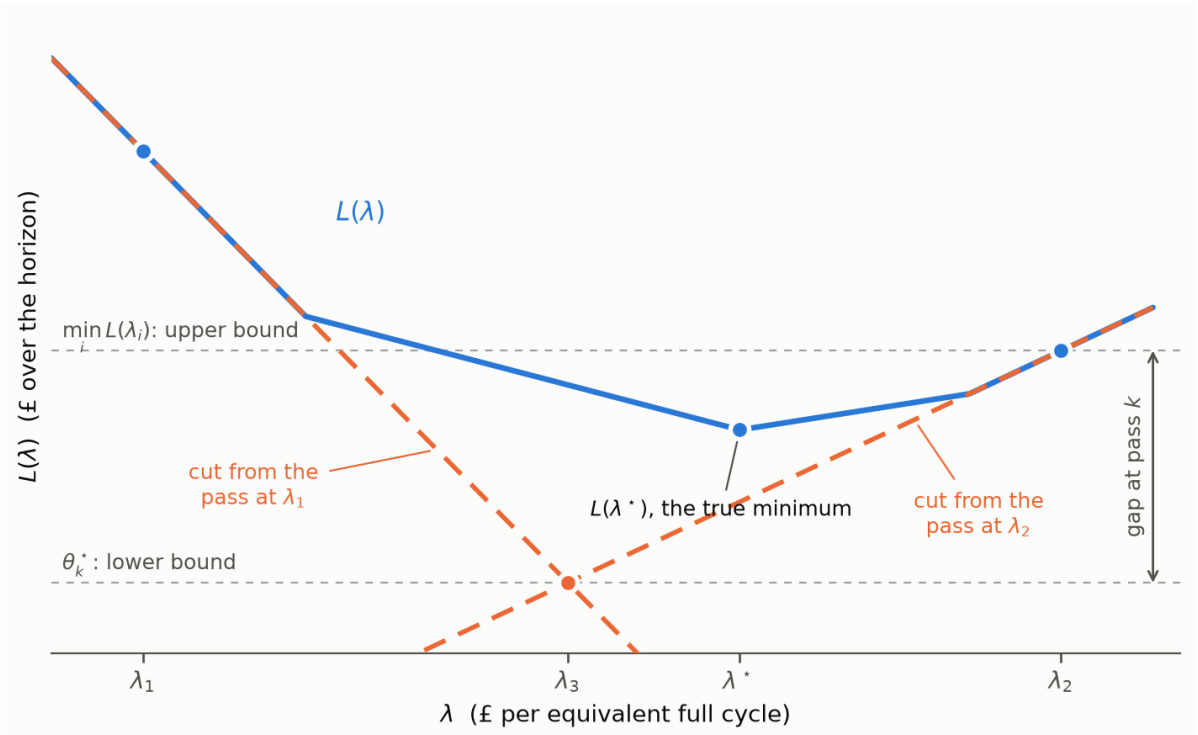


Figure 6.1: The dual function and two cutting planes: a convex piecewise-linear $L(\lambda)$, the cuts generated by the passes at λ_1 and λ_2, their crossing at λ_3 giving the lower bound $\theta_k^\star$, and the gap up to the best value evaluated so far.

> the instances tried, but §4.6's time-varying allowance $\overline{L}(t)$ would need one multiplier per quota period, with $g_y = \overline{L}_y -$ (cycles spent in period y) and the same master program widened to Y columns. That extension is the main structural argument for this method over a one-dimensional search, which cannot represent it at all.

One caution belongs with the theory rather than with any measurement: the Lagrangian relaxation of a *mixed-integer* program generally leaves a duality gap, so $\min_\lambda L(\lambda)$ may exceed $P^\star$ strictly. The method remains correct, returning a schedule and a valid bracket, but the bracket does not close. Weak duality also requires each evaluation to be a true maximum of the relaxed problem, which a rolling pass with finite lookahead only approximates, so a bound quoted from decomposed evaluations is sound only as far as that approximation holds.

Both methods are implemented and both are selectable with `battery-case dispatch --cycle-pricer {cutting-plane,bisection}`, cutting planes being the default: it costs fewer passes and returns the bound above. They share one evaluation oracle and differ in the subproblem they ask it for: bisection reads one bit from a *capped* pass, the cutting-plane method a value and a subgradient from a *relaxed* one. The reason a capped pass admits no usable subgradient, the measured comparison of the two against exactly solved instances, and the validation of the bound are in `cycle-price-experiments.md`; the architecture of the seam is in `architecture.md`.

6.7 The cycle budget on this data

With the full 5,000-cycle budget of §5 the *cap is never reached* and $\lambda^\star = 0$: the round-trip loss of §4.2 rules out the narrow spreads long before the cycle budget runs out, so the windows need no rationing and the decomposition loses only the foresight past its lookahead. Unrationed dispatch spends 3,443 cycles (§7), so under the pro-rated budget of 1,500 cycles the cap *would*

bind and λ would become active; what it is worth there has not been measured, since each pass re-solves the whole horizon.

6.8 How much foresight is enough

One day of lookahead is sufficient on this data. Over June 2018:

Committed	Lookahead	Profit	Cycles
1 day	none	£4,589	78.4
1 day	1 day	£4,621	79.2
1 day	2 days	£4,621	79.2
7 days	1 day	£4,621	79.2

Deciding each day blind to the next costs 0.7%, and nothing is gained past the first day of lookahead: *committing a whole week at a time reaches the same profit as committing one day with one day in view.* That is what one would expect of a 4 MWh store that *can fill in two hours: its decisions do not reach far beyond the day they are taken in*, and the cycle budget, the one thing that would couple distant days, is not binding.

Chapter 7

Results

The figures for 2018–2020, and how to read them, are in `results.md`.

Part II

Battery dispatch under stochastic prices

Formulation of the battery dispatch problem when the two price series are not known in advance, under a day-ahead commitment structure: a multistage stochastic mixed-integer linear program whose deterministic counterpart is `model.md`.

Throughout, **M§4.2** means §4.2 of that document. This file states the model only. Nothing here is implemented, and no solution method is proposed; §10 says what is deferred and why.

The formulation rests on five modelling choices, stated once here and used everywhere below:

Choice	Meaning
a stage is a day, $W = 48$	prices are re-forecast daily; the tree branches once a day
both markets are committed the day before	every volume for day k is chosen at the end of day $k-1$
the objective is expected profit	risk neutral, no tail term
the cycle budget holds almost surely	it is a physical limit, not an average
the terminal stock is free	as in M§4.2

Together they buy a considerable simplification, set out in §4.2: the program reduces to a **deterministic** MILP on the tree, in which each node carries the conditionally expected prices of the following day. §4.3 says exactly which of the five choices that reduction depends on.

Chapter 1

What changes, and what does not

M§1 takes both price series as exogenous **and known**: the whole horizon is given before any decision is made, so the schedule is a single sequence chosen with perfect foresight. The battery remains a price taker here, and the series remain exogenous, but they are no longer known. *A volume delivered on day k must be chosen using only what has been observed up to the end of day $k-1$.*

Two things follow:

- The decision is no longer a schedule but a **policy**: a rule mapping observed history to a day's plan. On a finite scenario tree a policy is represented by one copy of the decision variables per node.
- The objective is a **functional of a distribution**, here the expectation.

The physics does not change. Every constraint of M§4.1 through M§4.7 holds in each node exactly as written, because none of them involves a price.

Chapter 2

Uncertainty and information

2.1 Probability space and filtration

Let $(\Omega, \mathcal{F}, \mathbb{P})$ carry the joint price process $\left(p_1(t), p_2(t)\right)_{t \in \mathcal{T}}$, and let

$$\mathcal{F}_{-1} \subseteq \mathcal{F}_0 \subseteq \cdots \subseteq \mathcal{F}_{K-1} = \mathcal{F}$$

be the filtration generated by observing prices day by day, with $\mathcal{F}_k$ the information available once day k has settled and $\mathcal{F}_{-1}$ trivial. The day-ahead commitment of §2.3 is the statement that the plan for day k is $\mathcal{F}_{k-1}$-measurable. That requirement is **non-anticipativity**, and it is the only structural addition to M§4.

2.2 The scenario tree

The filtration is represented by a finite tree, one level per day, with a root that carries no prices because nothing has been observed before day 0.

Symbol	Meaning
$k = 0, \ldots, K-1$	day
$\mathcal{N}_k$	the nodes carrying day k's prices; $\mathcal{N} = \{r\} \cup \bigcup_k \mathcal{N}_k$
r	the root, the state before day 0
$a(n)$	the parent of node $n \neq r$
$\mathcal{C}(n)$	the children of n
$\mathcal{L} = \mathcal{N}_{K-1}$	the leaves, one per scenario
$\mathcal{D} = \mathcal{N} \quad \mathcal{L}$	the **decision nodes**: those with children
$\pi(n) > 0$	the probability of reaching n

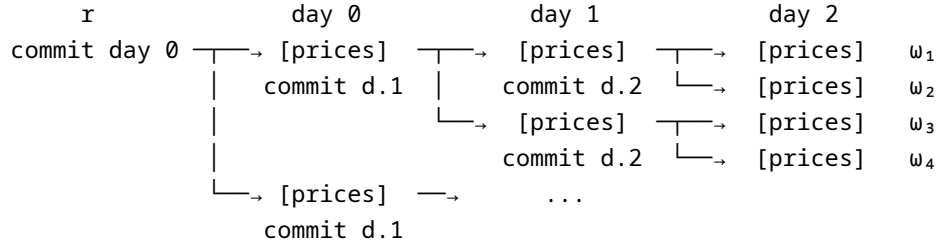
with $\pi(r) = 1$ and $\pi(n) = \sum_{n' \in \mathcal{C}(n)} \pi(n')$, hence $\sum_{n \in \mathcal{N}_k} \pi(n) = 1$ for every day k. A node is an information state: two scenarios sharing a node are indistinguishable up to that day.

Each node $n \neq r$ carries the realised price vectors $p_1(n, \tau)$ and $p_2(n, \tau)$ for the $\tau = 1, \ldots, 48$ settlement periods of its day. These are **data of the node**, not variables, which is what keeps the program linear. Because a node holds the joint realisation of both markets, any dependence between p_1 and p_2, contemporaneous or lagged, is represented exactly by the tree and needs no separate correlation structure.

2.3 Who decides what, and when

Both markets have gate closure on the previous day. The plan for a day is therefore chosen at that day's **parent**, and is necessarily the same for all children of that parent, since they are indistinguishable at the moment of commitment. Attaching the decision to the node where it is taken rather than to the node where it is delivered makes non-anticipativity automatic and removes the need for any explicit measurability row:

> For each $n \in \mathcal{D}$, the variables of §3 indexed by n are the plan for the day following n, committed knowing n's history, and delivered under each child's prices.



decisions and state live on □; prices live on the children

Leaves carry prices but no decisions: their day was committed at their parent. The root carries decisions but no prices: day 0 is committed under the prior.

Chapter 3

Notation

Sets and parameters of M§2 carry over unchanged, with $\Delta t = \frac{1}{2}$ h and $W = 48$ periods in a day. Added:

Symbol	Unit	Meaning
K	days	horizon in days; $T = 48K$
$\pi(n)$	–	probability of node n
$\bar{p}_m(n,\tau)$	£/MWh	conditionally expected price, §4.2

Every decision variable of M§2 gains a decision-node index and a within-day period index. For $n \in \mathcal{D}$ and $\tau = 1, \dots, 48$:

Symbol	Domain	Unit	Meaning
$c_1(n,\tau),\ c_2(n,\tau)$	≥ 0	MW	charging power bought from each market
$d_1(n,\tau),\ d_2(n,\tau)$	≥ 0	MW	discharging power drawn from storage for each market
$S(n,\tau)$	≥ 0	MWh	energy stored at the end of the period
$u(n,\tau)$	binary	–	1 when charging, 0 when discharging
$S_{\max}(n,\tau)$	≥ 0	MWh	effective storage volume after degradation
$n_{\text{cyc}}(n,\tau)$	≥ 0	cycles	equivalent full cycles accumulated along path(n)

u is **not** a recourse variable here: like every other volume it is fixed the day before, so the day's charge/discharge pattern is committed in full.

Two conventions carry the state across a day boundary. For every $n \in \mathcal{D}$ and every state variable $v \in \{S, n_{\text{cyc}}\}$,

$$v(n,0) \ := \ \begin{cases} v(a(n),48) & n \neq r \\ 0 & n = r \end{cases}$$

so that $S(r,0) = 0$ and $n_{\text{cyc}}(r,0) = 0$ reproduce the initial conditions of M§4.2 and M§4.6.

Chapter 4

Objective

4.1 Expected profit

Revenue is M§3, evaluated at the prices the delivering node turns out to pay. For $n \in \mathcal{D}$ and a child $m \in \mathcal{C}(n)$,

$$R(n,m,\tau) \;=\; \Delta t\Big[\,p_1(m,\tau)\big((1-e_d)\,d_1(n,\tau)-c_1(n,\tau)\big) \;+\; p_2(m,\tau)\big((1-e_d)\,d_2(n,\tau)-c_2(n,\tau)\big)\Big]$$

and the expectation over the tree weights each delivery by its probability:

$$\max\;\; \mathbb{E}\Big[\textstyle\sum_{t\in\mathcal{T}} R(t)\Big] \;=\; \max\;\; \sum_{n\in\mathcal{D}}\;\sum_{m\in\mathcal{C}(n)} \pi(m)\sum_{\tau=1}^{48} R(n,m,\tau)$$

The equality is the tower property applied to the tree: it is what makes the expectation a single linear objective over all node copies rather than a nested one.

4.2 The objective collapses onto conditional mean prices

Within the inner sum only the prices depend on m; the volumes are the same for every child, because they were committed before the child was known. Define the **conditionally expected price** of the day following n,

$$\bar{p}_m(n,\tau) \;=\; \sum_{m'\in\mathcal{C}(n)} \frac{\pi(m')}{\pi(n)}\; p_m(m',\tau), \qquad m \in \{1,2\}$$

Substituting and collecting terms gives the objective in its working form:

$$\max\;\; \sum_{n\in\mathcal{D}} \pi(n) \sum_{\tau=1}^{48} \Delta t\Big[\,\bar{p}_1(n,\tau)\big((1-e_d)\,d_1(n,\tau)-c_1(n,\tau)\big) + \bar{p}_2(n,\tau)\big((1-e_d)\,d_2(n,\tau)-c_2(n,\tau)\big)\Big]$$

No price uncertainty remains anywhere in the program. Each decision node sees a single price vector, its conditional mean, and the realisations enter only through that mean. This is **certainty equivalence**, and it holds for three reasons together: the decisions are deterministic given the node, revenue is linear in the prices, and no constraint contains a price. It is the reason the rest of this document is short.

What survives of the uncertainty is the tree itself. Two nodes at the same day with different histories hold different conditional means and therefore plan differently, which is precisely what makes the solution a policy rather than a schedule.

Chapter 5

Constraints

Each of the following holds **for every decision node** $n \in \mathcal{D}$ **and every** $\tau = 1, \dots, 48$ unless stated otherwise. Where a constraint is M§4 unchanged, that is said rather than re-argued.

5.1 Storage bounds

M§4.1, per node.

$$0 \;\le\; S(n,\tau) \;\le\; S_{\max}(n,\tau)$$

5.2 Energy balance within a day

M§4.2, per node, with the recursion running over the day's periods.

$$S(n,\tau) \;=\; S(n,\tau-1) + \Delta t \left[(1-e_c)(c_1(n,\tau)+c_2(n,\tau)) - d_1(n,\tau) - d_2(n,\tau)\right]$$

5.3 State handover at a day boundary

The convention of §3 states this as a definition rather than a row, and it is the only coupling between days:

$$S(n,0) = S(a(n),48), \qquad n_{\text{cyc}}(n,0) = n_{\text{cyc}}(a(n),48) \qquad \forall n \in \mathcal{D},\; n \neq r$$

Every child opens from the **same** physical state, because the plan that produced it was committed before the children were distinguishable. The scenarios diverge in their prices, not in where the battery starts, and not in what it did yesterday.

5.4 Complementarity and rate limits

M§4.3 and M§4.4, per node, with its own binary.

$$0 \;\le\; c_1(n,\tau) + c_2(n,\tau) \;\le\; \overline{C}\, u(n,\tau)$$
$$0 \;\le\; d_1(n,\tau) + d_2(n,\tau) \;\le\; \overline{D}\,(1-u(n,\tau))$$

5.5 Market 2 hourly commitment

M§4.5, within the day. Writing $\mathcal{T}_h$ for the two periods of hour h,

$$c_2(n,\tau) = c_2(n,\tau'), \qquad d_2(n,\tau) = d_2(n,\tau') \qquad \forall\, \tau,\tau' \in \mathcal{T}_h,\ \forall h = 1,\dots,24$$

A day is a whole number of hours, so no hour is split across nodes and this is the whole of M§4.5. Note that it is a statement about *granularity* and remains necessary: day-ahead commitment fixes when the volume is chosen, not that Market 2 is sold in hourly blocks while Market 1 is sold by the half hour.

5.6 Cycle accounting and cycle life

M§4.6's accumulation, per node, on the discharge side only.

$$n_{\text{cyc}}(n,\tau) \;=\; n_{\text{cyc}}(n,\tau-1) + \frac{\Delta t\,\big(d_1(n,\tau)+d_2(n,\tau)\big)}{\overline{S}_{\max}}$$

The count is path dependent: $n_{\text{cyc}}(n,48)$ is the throughput along the path from r to n and nothing else. The budget holds on every realisation,

$$n_{\text{cyc}}(n,\tau) \;\le\; \overline{L} \qquad \forall n \in \mathcal{D},\ \forall \tau$$

Cycle life is a physical property of the asset, not an average over futures that did not happen. As in M§4.6, n_{cyc} is non-decreasing along a path, so capping each node caps every scenario: because every scenario's total is attained at the last decision node on its path, the per-node form above and the per-scenario form $n_{\text{cyc}}(a(\omega),48) \le \overline{L}$ for all $\omega \in \mathcal{L}$ define the same feasible set.

A λ priced against this cap is its shadow price, with the same interpretation as M§6: one multiplier per node, each the marginal value of a cycle on that history.

5.7 Storage volume degradation

M§4.7, per node, linear in the node's accumulated count.

$$S_{\max}(n,\tau) \;=\; \overline{S}_{\max}\,\big(1-\rho\, n_{\text{cyc}}(n,\tau)\big)$$

As in M§4.7 the variable may be substituted out of §5.1, leaving one row per node and period in S and n_{cyc}. Because n_{cyc} is path dependent, so is the usable volume: two nodes of the same day hold batteries of different size, which is a real effect of the policy and not an artefact of the tree.

5.8 Terminal condition

$S(n,48)$ is left free at the last decision nodes, as M§4.2 leaves $S(T)$ free. This is defensible over a horizon that is short beside the asset's life, and it is what keeps §4.2's collapse intact; the cost is that the policy is rewarded for emptying the store at the end of the horizon, so a long rolling study should either impose $S(n,48) = S(r,0)$ or attach a terminal value, and accept the consequences listed in §4.3.

Chapter 6

The deterministic equivalent, and its size

§4 and §5 together are one mixed-integer linear program over the variables of all decision nodes at once. By §4.2 it is a MILP with *no* stochastic element left in its rows: the tree supplies the branching and the conditional means, and everything else reads exactly like M§4.

Its size is the tree's. With uniform branching b at each day,

$$|\mathcal{N}| \;=\; \frac{b^{K+1}-1}{b-1}, \qquad |\mathcal{L}| \;=\; b^{K}, \qquad |\mathcal{D}| \;=\; \frac{b^{K}-1}{b-1}, \qquad \text{binaries} \;=\; 48\,|\mathcal{D}|$$

so the program grows geometrically in the horizon:

Days K	Branching b	Decision nodes	Binaries
7 (one week)	2	127	6,096
7 (one week)	3	1,093	52,464
14 (fortnight)	3	2,391,484	114,791,232
1,096 (three years)	2	$2^{1096}-1$	—

The second row is the useful comparison in one respect only: a **one-week** tree that branches three ways a day carries as many binaries as the **three-year** deterministic program of M§6. It should not be read as implying the same difficulty. That program's hour was spent entirely at the root (`tractability.md`: `Nodes 0`, the search never branched), on a relaxation weakened by a cycle budget coupling 1,096 days; over a week that budget does not bind and the paths are short. The point of the table is the exponent, not the row: a tree over three years of daily stages has more nodes than can be enumerated, whatever the hardware and whatever the decomposition, which is why a policy must be computed on a short tree and rolled.

Two consequences, both structural rather than computational:

- The deterministic equivalent states the problem; it is not the form in which the problem would be handed to a solver. Nothing above says a tree of useful size is out of reach, only that reaching it monolithically is the wrong route. It is worth writing down because it fixes exactly what a solution method must reproduce.

- The horizon and the tree are separate approximations. A policy may be computed on a short tree and applied in a rolling fashion over the long horizon, in which case the tree represents the uncertainty that matters for the commitment being made tonight, and the handover of §5.3 carries the rest. Under that scheme the tree built tonight spans the few days over which the conditional mean still moves with what was observed today, not the study horizon, and is correspondingly small.

6.1 What a decomposition would have to respect

The program is already structured for decomposition, and in the same way the deterministic model is: days couple only through the two-dimensional state (S, n_{cyc}) of §5.3. Relax that handover and what is left is one 48-period MILP per decision node, each the size of a single day and each solved in milliseconds. Two families exploit this, and they split the program along different seams:

- **By node, along the state.** Treating (S, n_{cyc}) as the state of a dynamic program is what makes the rolling decomposition of `tractability.md` work on the deterministic model, and the tree is its stochastic analogue: a value function per day, estimated from the children. Stochastic dual dynamic programming is the standard method here, and it is **not valid as it stands**, because the $u(n, \tau)$ of §5.4 are integer and the value function over the state is therefore not convex. Making it valid means Lagrangian cuts over a binarised state, as in SDDiP, or accepting an approximation whose error is not bounded by the method.
- **By scenario, along the commitment.** Relaxing the split-variable rows of §9 leaves one copy of `model.md` per scenario, each solvable by the means already in use. Progressive hedging and dual decomposition both work this way. Neither terminates with a certificate on a mixed-integer program: they return a primal schedule and a dual bound, and the gap between them is the answer's quality rather than a proof of optimality.

The distinction that matters is therefore not whether the deterministic equivalent can be solved whole — it cannot, at any interesting size — but that the integers of §5.4 cost the convexity the cheaper decompositions rely on. The day-ahead commitment does not help here: it removes the integers' *recourse* character, so no binary is ever chosen after a price is seen, but the binaries remain in every node's program and so does the non-convexity of the value function over the state.

Chapter 7

Special cases and checks

Each is a test a correct implementation would have to pass.

It collapses to the deterministic model. With $b = 1$ at every day the tree is a single path, $\bar{p}_m(n, \cdot)$ is the next day's actual price, and §4 to §5 become M§3 to M§4.7 exactly, with the same optimum. Perfect foresight is the degenerate case of this model, not a different one.

Stage-independent conditional means make the tree redundant. If $\bar{p}_m(n, \cdot)$ depends only on the day and not on the history, as under an i.i.d. price process, every node of a given day carries the same data and opens from a state its own plan produced; an optimal solution then exists that is the same at all nodes of a day, and the tree collapses to a single path of K days run on mean prices. In that case the stochastic model and the deterministic model on the mean path have the same value, so VSS $= 0$ below. All of the value of this formulation lies in how much the conditional mean of tomorrow moves with what was seen today.

It is bounded by perfect foresight. Solving `model.md` separately on each scenario and averaging gives the wait-and-see value

$$\mathrm{WS} \;=\; \sum_{\omega \in \mathcal{L}} \pi(\omega)\, V_{\mathrm{det}}(\omega), \qquad \mathrm{EVPI} \;=\; \mathrm{WS} - \mathrm{RP} \;\geq\; 0$$

where RP is the optimum of §4 to §5 and $V_{\mathrm{det}}(\omega)$ the deterministic optimum under ω. EVPI is what foresight is worth and therefore the ceiling on what any better forecast could add. Under this commitment structure it includes the value of knowing *today's* prices before committing today's volumes, which is information the policy never has.

It is bounded below by the deterministic solution it replaces. Solve `model.md` once on the unconditional mean price path, fix the committed decisions, evaluate the expectation of the rest, and call the result EEV. Then

$$\mathrm{VSS} \;=\; \mathrm{RP} - \mathrm{EEV} \;\geq\; 0$$

the value of the stochastic solution: what is gained by modelling the uncertainty rather than its average. VSS is the number that decides whether this formulation is worth solving at all.

Chapter 8

Assumptions this formulation adds

Beyond those of M§5, which all carry over:

- **The price process is exogenous and the battery remains a price taker.** The tree is fixed before optimisation; no decision changes a probability or a price. Without this the objective would be bilinear and the program would stop being a MILP.
- **A finite tree represents the filtration.** The continuous price process is replaced by finitely many realisations with known probabilities. The tree is an input here, and its construction, whether by sampling, by moment matching or by a lattice, is not part of this formulation. Any quantity computed from it is conditional on it. By §4.2 only the conditional first moments of the children enter the objective, which is a weak demand on the tree and should not be mistaken for a licence to build it carelessly: the constraints still see the branching, and §7 shows the whole value of the model lies there.
- **Both markets close the day before, and the day is the unit of re-planning.** A gate closure falling inside a day cannot be represented without splitting the day, and the battery cannot react within the day to a price it did not expect, not even by switching between charging and discharging.
- **The state is the pair $(S, n_{\mathbf{cyc}})$ and nothing else.** §5.3 carries exactly these across a boundary, as in M§6. If degradation acquired a calendar component, or the cycle count could recover with rest, the state would widen and §5.3 with it.

Chapter 9

The scenario form

An equivalent statement indexes variables by scenario and imposes non-anticipativity explicitly. Let every $\omega \in \mathcal{L}$ carry its own full-horizon copy $x(\omega)$ of the variables of M§2, subject to all of M§4 along that scenario's prices. The program is then

$$\begin{aligned} \max \quad & \sum_{\omega \in \mathcal{L}} \pi(\omega)\, Z(x(\omega)) \\ \text{s.t.} \quad & x(\omega) \in \mathcal{X} \qquad \forall \omega \in \mathcal{L} \\ & x_k(\omega) = x_k(\omega') \qquad \forall k,\ \forall \omega, \omega' \text{ sharing the day-}(k-1)\text{ node} \end{aligned}$$

where $\mathcal{X}$ is the deterministic feasible set of M§4, the same for every scenario because no constraint involves a price, and the last line is the **split-variable** constraint, lagged by one day to match §2.3.

The two forms have the same optimum. The node form (§4 to §5) is smaller, having one copy per decision node rather than one per scenario per day, and is what a direct solve would use. The scenario form is larger but separable once the coupling row is relaxed, each scenario then being exactly the program of `model.md`, which is the structure progressive hedging and scenario decomposition exploit, as §6.1 sets out.

Chapter 10

What is deferred

- **How the tree is built.** Scenario generation, reduction and the testing of whether a tree represents the price process are prior to this formulation and independent of it.
- **How it is solved.** §6 shows the deterministic equivalent is beyond direct solution at any interesting size, and §6.1 says which seams a decomposition would open and what the integers of §5.4 cost it. Choosing between those methods, implementing one and establishing the quality of the policy it returns are all outside this document.
- **What it is worth.** VSS and EVPI in §7 are definitions, not measurements. On this data the deterministic model already reaches within 0.41% of its own optimum (`tractability.md`) and one day of foresight captures nearly all of the value available to it (`model.md` §6), so the quantity to establish first is whether VSS is large enough to justify any of the above.